PODCAST STUDIO / TEST DOCUMENT

Understanding AI Systems: From Machine Learning to Agents

Sample lesson notes | English | Text-based PDF for Lesson Recap and Exam Preparation

Learning objective

By the end of this lesson, a student should be able to distinguish traditional machine learning from generative AI, explain how retrieval-augmented generation works, and describe when an AI workflow qualifies as an agent.

1. What is machine learning?

Machine learning (ML) is a way to build systems that learn useful patterns from data rather than relying only on fixed instructions written by a programmer. In **supervised learning**, a dataset contains examples with known target labels. For example, a spam classifier can learn from messages marked as “spam” or “not spam.” The model uses features of new messages to predict their labels. To evaluate performance honestly, teams test the model on examples it did not train on. A model that performs well only on training data may fail to generalize to real situations.

2. How is generative AI different?

A conventional classifier might answer “spam” or “not spam.” **Generative AI** instead produces new content such as text, images, code, or audio. A large language model (LLM) generates text by working with tokens and predicting plausible continuations in context. This makes LLMs useful for drafting, explanation, translation, and summarization. However, fluent output is not the same as verified truth: an LLM can confidently generate incorrect details. For learning applications, the source material should remain the basis for factual claims, and important answers should be checked.

3. Prompting and context

Prompts shape what an LLM should do. A system-level instruction can establish the role, tone, restrictions, and output format, while the user input supplies the specific task and source text. A useful educational prompt might ask for five key ideas, short examples, and a concise summary. A different prompt may ask for three self-check questions with answers. The two tasks can use the same underlying notes but produce different learning experiences. Even a strong prompt cannot guarantee factual correctness, so the application should keep the original source available for review.

Core distinction: predicting a category, generating a summary, and taking an action are three different system behaviors.

From answers to useful workflows

4. Retrieval-augmented generation (RAG)

Retrieval-augmented generation combines a language model with a way to retrieve relevant information from a document store, database, or other external source. Imagine a university assistant answering questions about current course policies. Instead of relying only on what the model learned during training, the system searches the latest handbook, selects relevant passages, and supplies those passages as context for the answer. This can improve relevance and grounding, but it does not automatically eliminate mistakes. Retrieval quality matters: if the wrong passage is selected, the final answer may still be wrong.

5. What makes an AI agent different?

An **AI agent** is generally designed to pursue a goal by choosing steps and using available tools. An agent may call an API, look up a record, or update a task after checking permissions. For example, a study-planning agent could inspect upcoming deadlines, identify an open study slot, and propose a schedule. Tool access needs boundaries: the system should not send messages, change calendars, or purchase items without appropriate authorization. By contrast, a simple fixed pipeline that reads notes, generates a script, and converts the script to audio is an **AI-powered workflow**; it does not become an autonomous agent just because it uses an LLM.

6. Example: a podcast learning assistant

Consider a two-stage educational application. First, a data processor extracts readable text from a PDF or a text file. Next, an LLM converts the cleaned notes into an audio-friendly script. Finally, text-to-speech produces an MP3 file, which the web interface lets the student play or download. A **Lesson Recap** mode explains the main points. An **Exam Preparation** mode adds exactly three review questions and answers grounded in the notes. This design makes it possible to test extraction, script creation, and speech synthesis independently, rather than rebuilding the whole application when one stage fails.

7. Quality checks and responsible use

A reliable AI application checks both software behavior and learning value. Unit tests can verify text cleaning, input validation, and that a selected mode sends the intended prompt. A live end-to-end test also checks the actual LLM response, the synthesized audio, and the downloadable file. Useful quality questions include: Are the main concepts accurate? Are examples supported by the notes? Is the narration understandable? Is the file playable? Finally, API keys should be stored outside public source code, and uploaded learning materials should be handled with privacy in mind.

Three ideas to remember

Supervised ML learns from labeled examples. **RAG** brings retrieved source information into an LLM response. **Agents** can select actions and use tools toward a goal, whereas a fixed LLM-to-audio pipeline is a workflow.